Authentication & Web Security Cheatsheet

Lessons: 56 — Authentication & Sessions · 57 — Web Application Security

Examples: 56 · 57

Covers: password hashing, sessions, JWT, OAuth2/OIDC, CSRF, injection, XSS, headers, TLS, uploads

Legend: `[*]` = API the lessons have not covered yet

PASSWORDS

```
bcrypt.GenerateFromPassword(pw, bcrypt.DefaultCost)    cost 10–12; 72-byte limit
bcrypt.CompareHashAndPassword(hash, pw)   constant-time by construction
argon2.IDKey(pw, salt, time, memory, threads, keyLen)  argon2id — the modern choice
                                          memory ~64MB, time 1–3, threads = cores
NEVER                          md5, sha1, sha256 — they're fast, which is the problem
NEVER                          your own salt-and-pepper scheme
salt                           per password, random, stored with the hash (both do this)
subtle.ConstantTimeCompare(a, b)     for comparing tokens/HMACs, not passwords
same error for both cases      "invalid credentials" — never "no such user"
dummy hash on unknown user     or the response time leaks which emails exist
rehash on login           [*] when the stored cost is below your current policy
```

TOKENS & RANDOMNESS

```
crypto/rand                    the ONLY correct source for anything security-related
rand.Read(b)                   fill a []byte with CSPRNG bytes
rand.Text()               [*] Go 1.24+: a random base32 string, ready to use
base64.RawURLEncoding.EncodeToString(b)   URL-safe, no padding
32 bytes                       the sane minimum for a session id or reset token
math/rand                      NEVER for tokens, ids, or salts — it's predictable
store a HASH of the token      so a database leak isn't a session leak
```

COOKIES & SESSIONS

```
http.SetCookie(w, &http.Cookie{
  Name: "session", Value: id,
  HttpOnly: true,              JavaScript cannot read it — the XSS mitigation
  Secure:   true,              HTTPS only
  SameSite: http.SameSiteLaxMode,     Lax is the sane default; Strict for banking
  Path:     "/",
  MaxAge:   3600,              or Expires; 0 means a session cookie
})
server-side session            the cookie holds an opaque id; the state lives in Redis/DB
                               — revocable instantly, which is the whole point
rotate the id on login         the fix for session fixation
delete on logout               server-side AND MaxAge: -1 on the cookie
idle + absolute timeout        both, not just one
__Host- prefix            [*] browser-enforced: Secure, Path=/, no Domain
```

JWT (built by hand)

```
header.payload.signature     three base64url parts joined by dots
header                       {"alg":"HS256","typ":"JWT"}
claims                       iss, sub, aud, exp, nbf, iat, jti
sign                         HMAC-SHA256(base64(header)+"."+base64(claims), secret)
verify                       recompute and compare with hmac.Equal — CONSTANT TIME
ALG CONFUSION                the classic attack: the token says "alg":"none" or
                             swaps RS256 for HS256 and signs with the public key.
                             THE FIX: decide the algorithm in YOUR code, never read
                             it from the token.
check exp AND nbf            with a small clock skew allowance
check aud and iss            a token for another service is not a token for you
never put secrets in a JWT   it is signed, NOT encrypted — anyone can read it
you cannot revoke a JWT      that's the trade-off: short expiry + a denylist for jti
access + refresh             access 5-15 min, refresh days, stored server-side
refresh ROTATION             a new refresh token each use; the old one is invalidated
reuse detection              an old refresh token reappearing = theft -> kill the family
session vs JWT               session for your own web app (revocable, simple)
                             JWT for stateless service-to-service and mobile
```

OAUTH2 & OIDC

```
authorization code + PKCE    the only flow for web and mobile apps today
  1. redirect to /authorize with client_id, redirect_uri, scope, state,
     code_challenge (S256 of a random verifier)
  2. user consents; the provider redirects back with ?code=&state=
  3. VERIFY state matches what you stored (CSRF on the callback)
  4. POST /token with the code + code_verifier -> access/refresh/id tokens
state parameter              random, single-use, tied to the session
PKCE                         proves the same client that started the flow finished it
implicit flow                dead; never use it
OIDC id_token                a JWT ABOUT the user; verify signature (JWKS), iss,
                             aud, exp, and the nonce
access token ≠ identity      it authorizes an API call; the id_token identifies
scopes are not permissions   your own authz still applies (see the RBAC sheet)
```

CSRF & 2FA

```
CSRF                         a third-party page makes the browser send YOUR cookie
SameSite=Lax                 blocks the common case, not all of it
double-submit cookie         a random token in a cookie AND in the form/header;
                             the server compares them
synchronizer token           the token lives in the server-side session — stronger
per-session, not per-request unless you need it; per-request breaks tabs
only for cookie auth         a Bearer header is not sent automatically -> no CSRF
check Origin/Referer     [*] defence in depth on state-changing requests
TOTP 2FA                     a shared secret + a 30s time step; allow ±1 step
                             store the secret encrypted; issue single-use recovery codes
login lockout                progressive delay or a lockout per account AND per IP
password reset               single-use, short-lived, hashed token; invalidate sessions
                             on reset; the same response whether the email exists or not
```

INJECTION

```
SQL                          parameterized queries, ALWAYS
                             db.Query("... WHERE id = $1", id)
                             never fmt.Sprintf; the ONLY safe interpolation is an
                             allowlisted column name for ORDER BY
command                      exec.Command("ls", "-l", dir) — an ARG SLICE, never a
                             shell string; never sh -c with user input
path traversal               filepath.Join(base, name) is NOT enough — "../.." works.
                             Clean, then verify the result still has the base prefix,
                             or use os.Root / http.ServeFile (which checks for you)
template injection           html/template escapes; text/template does NOT
LDAP/NoSQL/header injection  same rule: never concatenate untrusted input into a
                             structured string
```

XSS & OUTPUT ENCODING

```
html/template                CONTEXT-AWARE auto-escaping: HTML, attribute, JS, URL,
                             CSS — the reason to use it over text/template
text/template                NO escaping — never for HTML
template.HTML(x)             "I promise this is safe" — the bypass; with user input
                             it IS the vulnerability
{{.}} in a <script>          html/template escapes it correctly; string concatenation
                             into JS does not
CSP                          the second line of defence
never build HTML with fmt.Sprintf
```

SECURITY HEADERS

```
Content-Security-Policy      "default-src 'self'" — start strict, loosen deliberately
Strict-Transport-Security    "max-age=31536000; includeSubDomains"
X-Content-Type-Options       "nosniff"
X-Frame-Options: DENY        or CSP frame-ancestors 'none'
Referrer-Policy              "strict-origin-when-cross-origin"
Permissions-Policy       [*] turn off camera/microphone/geolocation
Cache-Control: no-store      on authenticated responses
set them in ONE middleware   not per handler
```

CORS & TLS

```
CORS is not security         it relaxes the browser's same-origin policy for YOU
Access-Control-Allow-Origin  an explicit allowlist; NEVER "*" with credentials
Allow-Credentials: true      requires an exact origin, not a wildcard
preflight                    answer OPTIONS with Allow-Methods/Headers/Max-Age
TLS floor                    &tls.Config{MinVersion: tls.VersionTLS12}
verify certificates          InsecureSkipVerify: true in production is a backdoor
HSTS + redirect              http -> https, and the header on every response
```

NOT TRUSTING THE REQUEST

```
r.Body = http.MaxBytesReader(w, r.Body, 1<<20)     cap it BEFORE reading
dec.DisallowUnknownFields()  reject overposting and typos
allowlist validation         enumerate what's allowed; never blocklist
validate types AND ranges    length, format, bounds, enum membership
SSRF                         user-supplied URLs: allowlist the host, resolve the DNS
                             and REJECT private ranges (127/8, 10/8, 169.254/16),
                             and disable redirects
open redirect                ?next=... must be a relative path or an allowlisted host
IDOR                         /orders/{id} — check OWNERSHIP, not just authentication
uploads                      sniff the content type (http.DetectContentType), cap the
                             size, generate your own filename, store outside the
                             webroot, never trust the client's Content-Type
webhooks                     verify the HMAC signature with hmac.Equal, and check the
                             timestamp to stop replays
```

SAFE FAILURE

```
generic errors to the client "something went wrong" + a correlation id
detail to the logs           stack, query, input — never to the response
redact secrets in logs       slog ReplaceAttr, centrally
no stack traces in responses  they name your files, your framework, your version
timeouts on the server       ReadHeaderTimeout stops Slowloris
rate limit auth endpoints    login, reset, register, token — before anything else
Go kills these for free      RE2 regexp = no ReDoS; net/http sanitizes header values
```

TRAPS & MEMORIZE

```
sha256 for passwords          a GPU tries billions per second
math/rand for tokens          predictable; every token is guessable
reading "alg" from the JWT    alg confusion — decide it in your code
no exp on a token             a stolen token is valid forever
JWT for your own web session  unrevocable, and you rebuilt cookies badly
InsecureSkipVerify            a deliberate MITM hole
"*" CORS with credentials     the browser refuses it; people then "fix" it by
                              reflecting the Origin — which is worse
filepath.Join with user input traversal; verify the prefix afterwards
text/template for HTML        no escaping at all
template.HTML on user input   the escaping you just bypassed
error messages that differ    user enumeration on login and reset
missing ownership check       authenticated ≠ authorized (IDOR)
trusting X-Forwarded-For      spoofable unless your proxy overwrites it
secrets in the repo/logs      the leak that outlives the incident
```